Universidad de Buenos Aires

Facultad de Ciencias Exactas y Naturales

TP2: AirMiles

Grupo JavaConMates

Integrantes:

Ayala Ignacio
Della Bonzana Lara
Morales Santiago
Vannelli Tiziana

5 de noviembre de 2026

1. Introducción

En este documento se justifican las estructuras de datos seleccionadas para implementar el TAD `AirMiles`. Para cada estructura se describe la información que almacena, las operaciones que permite realizar y su relación con las complejidades temporales esperadas.

Denotamos por U la cantidad de usuarios registrados, por P la cantidad de promociones y por $|L|$ la longitud del nombre de una promoción. Además, N representa la cantidad de usuarios solicitados para el ranking.

El diseño busca que las operaciones de consulta y modificación accedan directamente a la información que necesitan, evitando recorridos innecesarios. Las complejidades que dependen del ABB se analizan distinguiendo el caso promedio del peor caso, ya que el árbol utilizado no realiza balanceo automático.

2. Estructuras elegidas

2.1. ABB

Para representar el conjunto de usuarios se eligió un árbol binario de búsqueda (ABB) ordenado por el ID de cada usuario. De esta manera, cada usuario puede localizarse a partir de su identificador sin necesidad de recorrer todos los usuarios registrados.

Como nuestra implementación mantiene el árbol balanceado, buscar un usuario e insertar uno nuevo tienen complejidad $O(\log U)$, donde U es la cantidad de usuarios registrados. Esta estructura también permite acceder al usuario correspondiente para realizar operaciones como consultar su saldo, acumular millas, canjear millas o transferirlas. En todos estos casos, la búsqueda inicial cuesta $O(\log U)$ y las modificaciones de los atributos del usuario cuestan $O(1)$, sin considerar otras estructuras que deban actualizarse.

Por lo tanto, el ABB permite cumplir con las complejidades requeridas para las operaciones de acceso y registro de usuarios.

2.2. Trie

Para representar las promociones se eligió un Trie, ya que permite almacenar y buscar nombres recorriendo sus caracteres. Cada nodo tiene un arreglo de hijos correspondiente a las letras del alfabeto y puede contener los datos de una promoción cuando se alcanza el final de su nombre.

Cada promoción almacena su factor multiplicador y, cuando corresponde, un tope de millas. En este último caso, también se registra cuántas millas se otorgaron hasta el momento mediante esa promoción, lo que permite controlar que no se supere el límite establecido.

Las operaciones de búsqueda, inserción y eliminación de una promoción tienen complejidad $O(|L|)$, donde $|L|$ es la longitud de su nombre. Esta complejidad depende de la cantidad de caracteres que se deben recorrer y no de la cantidad total de promociones almacenadas.

El Trie se utiliza, entre otras operaciones, al acumular millas, ya que permite obtener la promoción correspondiente y calcular las millas que se otorgarán según su factor y su tope, así como también iniciar o finalizar una promoción, con o sin tope. Por eso, la búsqueda de la promoción durante una acumulación agrega un costo de $O(|L|)$.

2.3. MaxHeap

Para obtener el ranking de usuarios según sus millas disponibles se incorporó un MaxHeap. En esta estructura, el usuario con mayor cantidad de millas disponibles se encuentra en la raíz, por lo que consultar el máximo tiene complejidad $O(1)$.

Insertar un usuario o extraer el máximo y actualizar la prioridad de un usuario tiene complejidad $O(\log U)$, ya que puede ser necesario restaurar la propiedad de heap recorriendo una cantidad de niveles proporcional a la altura de la estructura.

Para poder actualizar eficientemente la prioridad de un usuario cuando cambia su saldo, cada usuario almacena su posición actual dentro del heap. Así, se accede directamente a su posición sin tener que buscarlo recorriendo todos los elementos, y luego se restaura la propiedad de heap en tiempo $O(\log U)$.

Para obtener los primeros N usuarios del ranking, se extrae el máximo sucesivamente y se guardan temporalmente los usuarios extraídos. Al finalizar, se los vuelve a insertar para conservar el contenido original del heap. Por lo tanto, obtener el ranking tiene complejidad $O(N \log U)$, suponiendo que N no supera la cantidad de usuarios registrados. Si se solicitan más usuarios de los que existen, se devuelven únicamente los disponibles.

2.4. ListaEnlazada

Para obtener todos aquellos pares que tienen relación exclusiva, se incorporó una ListaEnlazada. Para ello, se decidió que los usuarios tengan estos atributos: *esExclusivo, esExclusivoCon, handlePar.* El handle permite eliminar la relacion de la lista si se deja de cumplir la condición de exclusividad.

Por otro lado, paresTransferidores tiene complejidad $O(U)$ pues recorre una vez la lista y construye una matriz? de ids, y como cada usuario puede pertencer una sola vez, entonces la cantidad de relaciones no supera $U/2$, por lo tanto la complejidad es de $O(U/2)$ que es lo mismo que $O(U)$

2.5. Atributos auxiliares: `maxAcumulador` y `maxCanjeador`

Para cumplir con las complejidades $O(1)$ requeridas para consultar al usuario que más millas acumuló y al que más millas canjeó o transfirió, se agregaron dos referencias auxiliares: `maxAcumulador` y `maxCanjeador`.

Ambas referencias apuntan directamente a los objetos de tipo `Usuario` que cumplen esas condiciones. De esta forma, no es necesario recorrer el ABB ni el heap para responder las consultas.

Cuando se registra el primer usuario, se inicializan ambas referencias. Luego, cada vez que una operación modifica las millas relevantes de un usuario, se compara su nuevo valor con

el del máximo almacenado y se actualiza la referencia si corresponde. Para `maxAcumulador` se consideran las millas totales acumuladas; para `maxCanjeador`, la suma de las millas canjeadas y transferidas, según el criterio de nuestra implementación.

La consulta de cualquiera de estos máximos tiene complejidad $O(1)$. La comparación y eventual actualización de la referencia también cuestan $O(1)$ una vez que se tiene acceso al usuario.

3. Resumen de complejidades

En la siguiente tabla se resumen las complejidades de las operaciones implementadas. Se considera U como la cantidad de usuarios registrados, $|L|$ como la longitud del nombre de una promoción y N como la cantidad de usuarios solicitados en el ranking.

Operación	**Complejidad**
Crear el sistema	$O(1)$
Registrar un usuario	$O(\log U)$
Registrar un usuario con Referidor	$O(\log U)$
Consultar saldo o categoría	$O(\log U)$
Acumular millas	$O(\log U + \|L\|)$
Canjear millas	$O(\log U)$
Transferir millas	$O(\log U)$
Insertar una promoción	$O(\|L\|)$
Buscar una promoción	$O(\|L\|)$
Eliminar una promoción	$O(\|L\|)$
Consultar el máximo acumulador	$O(1)$
Consultar el máximo canjeador	$O(1)$
Obtener los primeros N usuarios del ranking	$O(N \log U)$
Obtener los pares de transferidores exclusivos	$O(U)$

Las complejidades de las operaciones sobre usuarios consideran el costo de buscar al usuario en el ABB. En las operaciones que modifican las millas disponibles también se debe actualizar su posición en el heap, lo que agrega un costo de $O(\log U)$ y no modifica el orden de complejidad indicado. En particular, acumular millas incluye además la búsqueda de la promoción en el Trie, por lo que su costo es $O(\log U + |L|)$.